

🔗 See Topic 05 → Binary Search Masterclass

🎯 **GOAL:** Master Java reference pointers, visualize every node reconnection before writing code, and eliminate `NullPointerException`s using Dummy Nodes!

🔧 REUSABLE LINKED LIST UTILITIES

Reverse Linked List Utility:

```
ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode nextTemp = curr.next;
        curr.next = prev;
        prev = curr;
        curr = nextTemp;
    }
    return prev;
}
```

Find Middle Node (Fast & Slow):

```
ListNode findMiddle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

💡 AHA MOMENT #1: THE DUMMY NODE SENTINEL PATTERN (FOR BEGINNERS)

Why do beginners get `NullPointerException` or write endless `if (head == null)` checks? Because the **head node can change or be deleted!** Creating a `ListNode dummy = new ListNode(-1); dummy.next = head;` anchors the list. Now perform all insertions/deletions on `dummy.next` safely, and return `dummy.next` as the new head!

1 JAVA REFERENCE VS ARRAY MEMORY

- **Array:** Contiguous memory block. $O(1)$ index lookup `arr[i]` via base address math.
- **Linked List:** Non-contiguous nodes scattered on Java Heap. Nodes link via reference pointers (`next`). $O(N)$ sequential traversal, but $O(1)$ node insertion/deletion once pointer is positioned!

2 WHY THIS VARIABLE EXISTS: 'LISTNODE TEMP'

💡 `ListNode temp = curr.next;`

Why it exists: Re-assigning `curr.next = prev` severs the forward arrow! Without saving `temp` first, nodes 3..N become completely unreachable, causing Java's Garbage Collector to erase the rest of your list!

🎨 ACTIVE LEARNING: DRAW IT YOURSELF #1

EXERCISE: Given list `1 → 2 → 3 → 4`. Draw the position of `prev`, `curr`, and `nextTemp` after exactly ONE iteration of reversing:
[Your Drawing Space: `prev = ____`, `curr = ____`, `nextTemp = ____`, List: `1 ← 2 → 3 → 4`]

🔗 VISUALIZING THE DUMMY NODE SENTINEL ANCHOR

WITH DUMMY NODE ANCHOR: `[dummy: -1] —next→ [Head: 10] —next→ [Node: 20] —next→ [Node: 30] —next→ null ▲` | `tail = dummy` (Perform all appends to `tail.next` safely! Return `dummy.next` at the end!)

🌟 **KEY TAKEAWAY:** Always use a Dummy Node whenever the head of the linked list might be modified, created, merged, or deleted!

📁

STEP-BY-STEP POINTER STATE TABLE (REVERSE LIST 1 → 2 → 3)

Step	prev	curr	nextTemp	Link State & List Representation
0 (Start)	null	1	2	1 → 2 → 3 → null (Original list)
1	1	2	3	null ← 1 2 → 3 → null (1st link reversed!)
2	2	3	null	null ← 1 ← 2 3 → null (2nd link reversed!)
3 (End)	3	null	null	null ← 1 ← 2 ← 3 (prev = 3) (Loop ends!)

1

DOUBLY LINKED LIST (DLL) NODE DELETION

DELETING TARGET NODE B FROM A ←→ B ←→ C: [A] ←prev—next→ [B] ←prev—next→ [C] STEP 1: Re-route A's next pointer to C: A.next = B.next; (A —next→ C)
STEP 2: Re-route C's prev pointer to A: C.prev = B.prev; (A ←prev— C) RESULT: [A] ←prev—next→ [C] (B is decoupled!)

📁

SINGLY LINKED LIST (SLL) VS DOUBLY LINKED LIST (DLL)

Operation	Singly Linked List (SLL)	Doubly Linked List (DLL)	Key Note
Memory per Node	1 Value + 1 Reference ('next')	1 Value + 2 References ('prev', 'next')	DLL consumes ~2x reference overhead
Delete Node given pointer	$O(N)$ (must find predecessor node)	$O(1)$ (via <code>node.prev.next = node.next</code>)	DLL is used in LRU Cache!
Backward Traversal	Impossible ($O(N)$ rebuild required)	$O(1)$ effortless via <code>curr.prev</code>	DLL allows bi-directional iteration

⚠️ POINTER ORDER MATTERS: Always connect `newNode.next` to the existing chain BEFORE re-routing `curr.next`, or you break the chain!

 PATTERN COMPOSITION: BUILDING COMPLEX SOLUTIONS

Complex Medium/Hard Problem	= Sub-Pattern 1	+ Sub-Pattern 2	+ Sub-Pattern 3
LC 143: Reorder List	Fast/Slow Find Middle	3-Pointer Reverse 2nd Half	Two-Pointer Interleave Links
LC 234: Palindrome List	Fast/Slow Find Middle	3-Pointer Reverse 2nd Half	Two-Pointer Value Comparison
LC 148: Sort List	Fast/Slow Split in Half	Recursive Divide & Conquer	Dummy Node Sorted Merge

1 FINDING MIDDLE NODE MECHANICS

```
public ListNode findMiddle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;    // 1 step
        fast = fast.next.next; // 2 steps
    }
    return slow; // Points to middle!
}
```

2 CYCLE START MATHEMATICAL PROOF

Let L = distance from Head to Entrance, k = distance from Entrance to Meeting Point.

$$\begin{aligned} \text{Distance}(\text{fast}) &= 2 * \text{Distance}(\text{slow}) \\ L + nC + k &= 2(L + k) \rightarrow L = nC - k \end{aligned}$$

 FAST & SLOW POINTER RACE VISUALIZATION

ODD LENGTH LIST (1 → 2 → 3 → 4 → 5 → null): Start: slow=1, fast=1 Step 1: slow=2, fast=3 Step 2: slow=3 (MID!), fast=5 → fast.next = null → STOP!

🔑 KEY TAKEAWAY: Break down complex medium problems into 3 basic building blocks: Find Mid + Reverse + Merge!

 PATTERN RECOGNITION KEYWORD QUICK-LOOKUP

Question Phrase Contains:	Target Linked List Pattern	Key Code Mechanism
"Reverse / Invert links in-place"	3-Pointer Reverse Window	<code>temp=curr.next; curr.next=prev; prev=curr; curr=temp;</code>
"Find Middle / Detect Loop"	Fast & Slow Pointers	<code>slow = slow.next; fast = fast.next.next;</code>
"Modify Head / Merge Lists"	Dummy Node Sentinel	<code>ListNode dummy = new ListNode(-1); dummy.next = head;</code>
"Remove N-th from End"	Fast & Slow N-Gap	Create gap of $N+1$ steps between fast and slow pointers
"Merge K Sorted Lists"	Min-Heap / PriorityQueue	<code>PriorityQueue<ListNode> minHeap</code> tracking active heads

1 ITERATIVE REVERSE LIST

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode nextTemp = curr.next; // Step 1: Save next
        curr.next = prev;              // Step 2: Reverse link
        prev = curr;                   // Step 3: Advance prev
        curr = nextTemp;               // Step 4: Advance curr
    }
    return prev; // New head!
}
```

2 MERGE TWO SORTED LISTS (DUMMY)

```
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(-1), tail = dummy;
    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) { tail.next = l1; l1 = l1.next; }
        else { tail.next = l2; l2 = l2.next; }
        tail = tail.next;
    }
    tail.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}
```

 **KEY TAKEAWAY:** Matching prompt keywords to pattern mechanics allows solving 90% of interview questions in under 15 minutes!

LINKED LIST INTERVIEW DECISION TREE (MERMAID)

PRINTABLE POINTER CHEAT SHEET

Pointer Name	Primary Purpose	When to Use	Common Mistake to Avoid
<code>dummy</code>	Anchor head node safely	Head can change/delete or merging lists	Returning <code>dummy</code> instead of <code>dummy.next</code>
<code>tail</code>	Tracks end of newly built list	Building or appending nodes dynamically	Forgetting <code>tail = tail.next</code> after appending
<code>prev</code>	Points to predecessor node	Reversing links or deleting target node	Initializing to <code>head</code> instead of <code>null</code> when reversing
<code>curr</code>	Points to active working node	Traversing or reversing current link	Advancing <code>curr = curr.next</code> after severing link
<code>nextTemp</code>	Saves forward link chain	Before executing <code>curr.next = prev</code>	Forgetting to declare <code>temp</code> (erases rest of list!)
<code>slow</code>	Moves 1 step at a time	Mid-finding, cycle detection, N-th end	Moving <code>slow</code> before <code>fast</code> initialization
<code>fast</code>	Moves 2 steps at a time	Mid-finding, cycle detection	Forgetting <code>fast.next ≠ null</code> check (NPE crash!)


FAANG COACH TIP: Print this page and review it 10 minutes before your technical interview!



1. LEETCODE 206 — REVERSE LINKED LIST

Easy ★★★★★ 3-Pointer Reverse Amazon Meta Google

★ FLOWCHART: prev=null, curr=head → Save temp=curr.next → curr.next=prev → Advance prev & curr

Problem:

Given head of singly linked list, reverse list and return new head.

Example:

1 → 2 → 3 → 4 → 5

→ Output:

5 → 4 → 3 → 2 → 1

💡 ALTERNATIVE SOLUTIONS:

1. **Stack:** Push all nodes to Stack, pop to rebuild ($O(N)$ Space).
2. **Recursive:** Reverses links on unwinding ($O(N)$ Call Stack).
3. **Iterative 3-Pointer:** $O(N)$ Time & $O(1)$ Space — **OPTIMAL CHOICE!**

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode nextTemp = curr.next; // Step 1: Save forward chain
        curr.next = prev;              // Step 2: Reverse link pointer
        prev = curr;                   // Step 3: Shift prev forward
        curr = nextTemp;               // Step 4: Shift curr forward
    }
    return prev; // Step 5: New reversed head!
}
```

2. LEETCODE 21 — MERGE TWO SORTED LISTS

Easy ★★★★★ Dummy Node Merge Amazon Microsoft

Problem:

Merge two sorted linked lists into one sorted list.

Example:

l1 = 1 → 2 → 4

l2 = 1 → 3 → 4

1 → 1 → 2 → 2 → 3 → 4 → 4

```
public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
    ListNode dummy = new ListNode(-1), tail = dummy;
    while (list1 != null && list2 != null) {
        if (list1.val <= list2.val) { tail.next = list1; list1 = list1.next; }
        else { tail.next = list2; list2 = list2.next; }
        tail = tail.next;
    }
    tail.next = (list1 != null) ? list1 : list2;
    return dummy.next;
}
```

3. LEETCODE 141 — LINKED LIST CYCLE

Easy ★★★★★ Fast & Slow Pointer Amazon Meta

Problem:

Determine if linked list has a cycle ('fast == slow').

```
public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true; // Cycle detected!
    }
    return false;
}
```

💡 AHA MOMENT (LC 21): Dummy Node sentinel completely removes edge case logic for choosing initial head node!

4. LEETCODE 876 — MIDDLE OF THE LINKED LIST

Easy
 ★★★★★
 Fast & Slow
 Google

Problem:
 Return middle node of linked list. If two middle nodes, return second middle.

```

public ListNode middleNode(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow; // Points to middle node!
}

```

Easy
 ★★★★★
 Mid + Reverse
 Meta
 Amazon

Problem:
 Check if linked list values form a palindrome in $O(N)$ time and $O(1)$ space.

?
 ALTERNATIVE SOLUTIONS COMPARISON:

1. **Array Copy:** Copy to ArrayList, use 2-pointer ($O(N)$ Space).
 2. **Stack:** Push 1st half, pop & compare ($O(N)$ Space).
 3. **Find Mid + Reverse 2nd Half:** $O(N)$ Time & $O(1)$ Space — **OPTIMAL!**

```

public boolean isPalindrome(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next; fast = fast.next.next;
    }
    ListNode secondHalf = reverse(slow), p1 = head, p2 = secondHalf;
    while (p2 != null) {
        if (p1.val != p2.val) return false;
        p1 = p1.next; p2 = p2.next;
    }
    return true;
}

private ListNode reverse(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode nextTemp = curr.next; curr.next = prev;
        prev = curr; curr = nextTemp;
    }
    return prev;
}

```

6. LEETCODE 19 — REMOVE NTH NODE FROM END OF LIST

Medium
 ★★★★★
 Fast/Slow Gap + Dummy
 Meta
 Amazon

Problem:
 Remove N -th node from end of list in one pass.

```

public ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0), fast = dummy, slow = dummy;
    dummy.next = head;
    for (int i = 0; i <= n; i++) fast = fast.next; // Create N gap
    while (fast != null) {
        fast = fast.next; slow = slow.next;
    }
    slow.next = slow.next.next; // Bypass N-th node!
    return dummy.next;
}

```

?
 AHA MOMENT (LC 19): Creating a gap of $N+1$ steps between `fast` and `slow` positions `slow` directly before the target node to delete!

7. LEETCODE 143 — REORDER LIST

Medium
★★★★★
Find Mid + Reverse + Interleave
Meta
Amazon

🔗 FLOWCHART: Find Mid → Reverse 2nd Half → Interleave L1 & L2 nodes alternatingly!

Problem:

Reorder list to $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$

Example:

1 → 2 → 3 → 4

→

1 → 4 → 2 → 3

.

```

public void reorderList(ListNode head) {
    if (head == null || head.next == null) return;
    // Step 1: Find Middle
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next; fast = fast.next.next;
    }
    // Step 2: Reverse Second Half
    ListNode prev = null, curr = slow.next;
    slow.next = null; // Split lists
    while (curr != null) {
        ListNode temp = curr.next; curr.next = prev;
        prev = curr; curr = temp;
    }
    // Step 3: Interleave Lists
    ListNode first = head, second = prev;
    while (second != null) {
        ListNode t1 = first.next, t2 = second.next;
        first.next = second; second.next = t1;
        first = t1; second = t2;
    }
}

```

8. LEETCODE 2 — ADD TWO NUMBERS

Medium
★★★★★
Dummy + Carry Digit
Amazon
Google

Problem:

Two non-empty linked lists representing two non-negative integers in reverse order. Add the two numbers and return sum list.

Example:

(2 → 4 → 3) + (5 → 6 → 4)

→

7 → 0 → 8

(342 + 465 = 807).

```

public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0), curr = dummy;
    int carry = 0;
    while (l1 != null || l2 != null || carry != 0) {
        int v1 = (l1 != null) ? l1.val : 0;
        int v2 = (l2 != null) ? l2.val : 0;
        int sum = v1 + v2 + carry;
        carry = sum / 10;
        curr.next = new ListNode(sum % 10);
        curr = curr.next;
        if (l1 != null) l1 = l1.next;
        if (l2 != null) l2 = l2.next;
    }
    return dummy.next;
}

```

💡 **AHA MOMENT (LC 143):** Reorder list combines 3 core sub-patterns: Fast/Slow Mid + 3-Pointer Reverse + Two-Pointer Interleaving!

9. LEETCODE 138 — COPY LIST WITH RANDOM POINTER

Medium
★★★★★
3-Pass Interleaving
Amazon
Meta
Google

Problem:
 Deep copy linked list where each node contains an additional `random` pointer in $O(N)$ time and $O(1)$ extra space!

```

public Node copyRandomList(Node head) {
    if (head == null) return null;
    // Pass 1: Interleave clone nodes (A → A' → B → B')
    Node curr = head;
    while (curr != null) {
        Node clone = new Node(curr.val);
        clone.next = curr.next; curr.next = clone;
        curr = clone.next;
    }
    // Pass 2: Assign random pointers
    curr = head;
    while (curr != null) {
        if (curr.random != null) curr.next.random = curr.random.next;
        curr = curr.next.next;
    }
    // Pass 3: Separate cloned list from original
    curr = head; Node dummy = new Node(0), copyCurr = dummy;
    while (curr != null) {
        copyCurr.next = curr.next; copyCurr = copyCurr.next;
        curr.next = curr.next.next; curr = curr.next;
    }
    return dummy.next;
}

```

10. LEETCODE 148 — SORT LIST (MERGE SORT ON LINKED LIST)

Medium
★★★★★
Divide & Conquer Merge
Meta
Google

Problem:
 Sort a linked list in $O(N \log N)$ time and $O(\log N)$ space complexity.

```

public ListNode sortList(ListNode head) {
    if (head == null || head.next == null) return head;
    // Step 1: Split list in half
    ListNode prev = null, slow = head, fast = head;
    while (fast != null && fast.next != null) {
        prev = slow; slow = slow.next; fast = fast.next.next;
    }
    prev.next = null; // Cut link
    // Step 2: Recursively sort halves & merge
    ListNode l1 = sortList(head), l2 = sortList(slow);
    return merge(l1, l2);
}

private ListNode merge(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0), tail = dummy;
    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) { tail.next = l1; l1 = l1.next; }
        else { tail.next = l2; l2 = l2.next; }
        tail = tail.next;
    }
    tail.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}

```

MEMORY BOX: 3-pass interleaving (`A → A' → B → B'`) achieves $O(1)$ space deep copying without needing a HashMap!

11. LEETCODE 23 — MERGE K SORTED LISTS

🔥 **Hard ★★★★★** **Min-Heap / PriorityQueue** [Amazon](#) [Meta](#) [Google](#)

Problem:
Merge K sorted linked lists into one sorted linked list in $O(N \log K)$ time!

```
public ListNode mergeKLists(ListNode[] lists) {
    if (lists == null || lists.length == 0) return null;
    PriorityQueue<ListNode> minHeap = new PriorityQueue<>((a,b) → a.val
    - b.val);
    for (ListNode node : lists) {
        if (node != null) minHeap.add(node);
    }
    ListNode dummy = new ListNode(0), tail = dummy;
    while (!minHeap.isEmpty()) {
        ListNode smallest = minHeap.poll();
        tail.next = smallest;
        tail = tail.next;
        if (smallest.next != null) minHeap.add(smallest.next);
    }
    return dummy.next;
}
```

12. LEETCODE 25 — REVERSE NODES IN K-GROUP

🔥 **Hard ★★★★★** **Group Reversal + Pointer Stitching** [Amazon](#) [Microsoft](#)

Problem:
Reverse nodes of list k at a time and return modified list.

```
public ListNode reverseKGroup(ListNode head, int k) {
    ListNode curr = head; int count = 0;
    while (curr != null && count != k) { // Find k-th node
        curr = curr.next; count++;
    }
    if (count == k) {
        ListNode reversedHead = reverseKGroup(curr, k); // Recurse rest
        while (count-- > 0) { // Reverse current k group
            ListNode temp = head.next; head.next = reversedHead;
            reversedHead = head; head = temp;
        }
        head = reversedHead;
    }
    return head;
}
```

🧠 **FAANG COACH TIP:** For Merge K Sorted Lists, Min-Heap maintains active head nodes in $O(\log K)$ per extraction!

★ THE 4 GOLDEN LAWS OF LINKED LISTS

- Dummy Node Anchor:** Use `ListNode dummy = new ListNode(-1); dummy.next = head;` whenever head node can change!
- Fast & Slow Speed Rule:** Move `slow = slow.next` (1x) and `fast = fast.next.next` (2x) for mid-finding & cycle detection.
- 3-Pointer Reverse Window:** Save forward chain first (`temp = curr.next`), reverse arrow (`curr.next = prev`), then shift window!
- Null Safety Check:** Always check `while (fast != null && fast.next != null)` before jumping 2 steps!

🚫 WHEN LINKED LIST FAILS (DO NOT USE!)

- Random Index Access:** Getting element at index i takes $O(N)$ sequential steps → Use **Array / ArrayList**!
- CPU Cache Locality:** Non-contiguous heap nodes cause high CPU cache misses → Arrays outperform Linked Lists for iteration speed!

📁 NEETCODE & BLIND 75 CONFIDENCE TRACKER

 Easy LC 206 Reverse List	 Easy LC 21 Merge Two Lists	 Easy LC 141 List Cycle
 Easy LC 876 Middle of List	 Easy LC 234 Palindrome List	 Med LC 19 Remove Nth End
 Med LC 143 Reorder List	 Med LC 2 Add Two Numbers	 Med LC 138 Copy Random
 Med LC 148 Sort List	 Hard LC 23 Merge K Lists	 Hard LC 25 Reverse K-Group

👉 **Next Master Topic: Topic 07** → **Stack & Queues Masterclass**

🚩 **TOPIC 06 LINKED LIST MASTERCLASS COMPLETE!** Turn the page to **Topic 07: Stack & Queues Masterclass!**